

Scoring Deep Dive (How Every Score Is Calculated)

This document explains, in implementation-level detail, how the backend computes:

- Repository **Code Quality** (including optional **Code Health** analysis)
- **Contribution** metrics and the **Commit Activity** score
- **Tech Stack** detection and scoring
- **Skill Authenticity** scoring (resume ↔ GitHub verification)
- **Profile Signal** scoring (portfolio completeness + activity)
- The final **Overall Score** and **Rating**

It is written to match the current code in `backend/services/`.

0) End-to-end data flow (what calls what)

The backend pipeline is roughly:

1. `resume_parser.py` extracts candidate info and a GitHub profile URL/username.
2. `github_service.py` fetches GitHub profile + repositories + aggregates.
3. `tech_stack_analyzer.py` detects technologies across repositories.
4. `skill_matcher.py` matches resume skills to GitHub-detected skills.
5. `code_quality_analyzer.py` grades repository quality and (optionally) code health.
6. `contribution_analyzer.py` summarizes commit activity and contributions.
7. `scoring_engine.py` combines all factors into a single weighted score.
8. `report_generator.py` formats a JSON report consumed by the frontend.

Important: GitHub API calls are rate-limited. Configure `GITHUB_TOKEN` in `.env` to reduce rate-limit issues.

1) Code Quality Analyzer (backend/services/code_quality_analyzer.py)

1.1 Inputs and outputs

Input:

- `repositories`: list of repo dicts from `GitHubService.fetch_repositories()` (name, stars, updated_at, pushed_at, is_fork, url, language, ...)
- `username`: GitHub login

Output: a dict containing:

- `overall_score` (0–100)
- `grade` (A/B/C/D/F) and `grade_label`
- `repositories_analyzed`
- `metrics` (portfolio-level averages)
- `detailed_analysis` (per-repo breakdown)
- `suggestions`, `strengths`, `weaknesses`

- `scoring_model` (notes on which optional deps were active)

1.2 Repository selection (which repos are analyzed)

Function: `_select_repos_for_analysis(repositories, max_repos=10)`

Steps:

1. Filter out forks:
 - keep repos where `is_fork == False`
2. Sort descending by a 2-key sort:
 - primary: `stars`
 - secondary: repo recency timestamp derived from `pushed_at` or `updated_at`
3. Take the top `max_repos` (default 10)

This means:

- A repo with more stars tends to be analyzed.
- If stars tie, the more recently pushed/updated wins.

1.3 Per-repository analysis (evidence collection)

Function: `_analyze_repository_quality(repo, username)`

For each selected repo:

1. Fetch root-level repository contents via GitHub Contents API:
 - `github_service.fetch_repository_contents(username, repo_name)`
2. Compute evidence signals from that root listing:
 - README presence/quality
 - docs folder presence (`docs/`, `doc/`, ...) **at repo root only**
 - presence of config (`.gitignore`, ...) **at repo root only**
 - presence of dependency manifests (`requirements.txt`, `package.json`, ...) **at repo root only**
 - examples/changelog/architecture doc presence **at repo root only**
 - contribution guide presence (`CONTRIBUTING.md`) **at repo root only**
 - code structure label (based on top-level directory names)
3. Fetch up to 30 commits (default) and compute commit frequency + message quality.
4. Optionally run "Code Health" analysis on real source files (Section 1.7).

Note: tests/CI/license signals are explicitly disabled in this version of the analyzer.

1.4 Category scores and weights (what categories exist)

Current included categories and weights:

- `documentation`: **0.15**
- `code_organization`: **0.30**
- `commit_quality`: **0.25**
- `code_health`: **0.30** (only when available and enabled)

Per-repo total uses `_weighted_category_score(scores)` which **re-normalizes** weights over the categories actually present.

Example:

- If `code_health` is missing/unavailable, the total is computed from only the remaining categories and their weights are re-normalized.

1.5 Documentation score (0–100)

Function: `_calculate_category_scores(evidence, repo) → scores['documentation']`

Components:

- If README exists:
 - `readme_quality_score * 0.65`
- If `docs/` (or similar root-level docs folder) exists:
 - `+18`
- If an architecture/design doc exists at root (e.g., `architecture.md`, `design.md`):
 - `+10`
- If examples/demo folder exists at root:
 - `+10`
- If README has a “usage” signal:
 - `+7`
- If `CONTRIBUTING.md` exists at root:
 - `+10`

Final:

- `documentation = clamp(doc_score, 0, 100)`

1.5.1 README quality score (structure + optional readability)

Function: `_evaluate_readme_quality(content)`

1. Structure score

Word-count contribution:

- `>= 500` words: `+35`
- `>= 250` words: `+25`
- `>= 120` words: `+15`
- `>= 60` words: `+8`

Section signals (regex-based):

- Installation/setup keywords: `+12`
- Usage/how-to/example keywords: `+12`
- Features/what-it-does keywords: `+10`
- Contributing/development keywords: `+8`
- Markdown badges pattern: `+5`

- Code blocks (``): +8

2. Readability score (optional; requires `textstat`)

If `textstat` is importable:

- compute Flesch Reading Ease: `textstat.flesch_reading_ease(content)`
- map to score:
 - readability ≥ 60 : +20
 - ≥ 40 : +15
 - ≥ 25 : +10
 - else: +6

3. Total:

- `readme_quality_score = clamp(structure_score + readability_score, 0, 100)`
- label mapping:
 - ≥ 80 : excellent
 - ≥ 60 : good
 - ≥ 35 : basic
 - > 0 : minimal
 - else: none

1.6 Code organization score (0–100)

This is based on **top-level directories**, not file contents.

1.6.1 Code structure label

Function: `_evaluate_code_structure(contents, dir_names)`

- Start with `structure_score = 0`
- For each “good pattern” dir at repo root, add +1:
 - `src`, `lib`, `app`, `core`, `utils`, `helpers`, `components`, `services`, `models`, `controllers`
- If repo has `src/` or `lib/` at root, add an additional +2.

Label mapping:

- `structure_score  $\geq 4$` → excellent
- `structure_score  $\geq 2$` → good
- else if any dirs exist → basic
- else → minimal

1.6.2 Code organization score formula

Function: `_calculate_category_scores(...) → scores['code_organization']`

- Base score by label:
 - excellent=100, good=75, basic=50, minimal=25
- Richness bonus:

- `min(15, directory_count * 1.5)`
- Final:
 - `code_organization = clamp(base * 0.85 + richness_bonus, 0, 100)`

1.7 Commit quality score (0–100)

Commit quality has 2 parts:

1. **Commit frequency** score (timeline dynamics)
2. **Commit message quality** score (clarity + conventions + anti-repetition)

Final category:

- `commit_quality = clamp(0.45*commit_frequency_score + 0.55*commit_message_score, 0, 100)`

1.7.1 Commit frequency score

Function: `_evaluate_commit_frequency(commits, repo)`

Inputs:

- `commits`: up to 30 commits (from GitHub API)
- Each commit has `.date` in ISO format if available

If fewer than 2 commit dates are available:

- fallback uses repo creation date:
 - `days_old = max((now - created_at).days, 1)`
 - `commits_per_month = (len(commits) / days_old) * 30`
 - `score = clamp((commits_per_month ** 0.6) * 18, 0, 100)`
 - label: `moderate` if score ≥ 45 else `low`

If 2+ commit dates exist:

1. Compute window and volumes:

- `window_days = max((newest - oldest).days, 1)`
- `commits_per_week = commit_count / max(window_days/7, 1)`
- `commits_per_month = commits_per_week * 4.345`

2. Recency ratio:

- `recent_30 = number of commits within last 30 days`
- `recency_ratio = recent_30 / commit_count`

3. Consistency:

- `intervals = day gaps between consecutive commits`
- `avg_gap = mean(intervals)`
- `gap_std = pstdev(intervals)`
- `consistency = 1 - clamp(gap_std / max(avg_gap, 1), 0, 1)`

- close to 1 → consistent spacing
- close to 0 → bursty/irregular

4. Subscores:

- `volume_score = clamp((commits_per_month ** 0.62) * 18, 0, 100)`
- `recency_score = clamp(recency_ratio * 100, 0, 100)`
- `consistency_score = clamp(consistency * 100, 0, 100)`

5. Weighted frequency score:

- `score = 0.55*volume_score + 0.25*recency_score + 0.20*consistency_score`

Label mapping:

- `>= 78 → very_active`
- `>= 62 → active`
- `>= 42 → moderate`
- `else → low`

Worked example (illustrative)

Suppose the analyzer sees 30 commits spanning 90 days:

- `window_days = 90`
- `commits_per_week ≈ 30 / (90/7) ≈ 2.33`
- `commits_per_month ≈ 2.33 * 4.345 ≈ 10.12`

Assume:

- 6 of 30 commits are in the last 30 days → `recency_ratio = 0.2`
- mean gap is 3 days, stddev gap is 2 days →
 - `consistency = 1 - clamp(2/3, 0, 1) ≈ 0.333`

Then:

- `volume_score = clamp((10.12 ** 0.62) * 18, 0, 100)`
- `recency_score = 0.2 * 100 = 20`
- `consistency_score ≈ 33.3`
- `final score = 0.55*volume + 0.25*20 + 0.20*33.3`

The exact result depends on the exponentiation, but the structure is:

- **volume dominates**, and
- recency + consistency adjust it up/down.

1.7.2 Commit message score

Function: `_evaluate_commit_messages(commits)`

For each commit's **first line**:

Positive signals:

- Good length (10–72 chars): +35
- Conventional commits prefix (feat/fix/docs/...): +30
- Otherwise, starts with a capital letter: +20
- Has issue reference (#123 or ABC-123): +10
- Starts with a “verb” like add/fix/refactor/...: +8

Negative signals:

- “Noisy” titles like `update`, `changes`, `misc`, `test`: -20

Per-commit score is clamped 0..100. Final message score is the mean over commits.

Then the analyzer subtracts a **repetition penalty** (Section 2) to avoid rewarding repeated/templated messages.

Worked example (illustrative)

Commit title: `feat(api): add patient lookup endpoint`

- length 10–72 → +35
- conventional prefix → +30
- issue ref? (none) → +0
- starts with verb? (starts with `feat(...)`, not checked as verb) → +0
- noisy? (no) → +0

Per-commit score: 65

Commit title: `Update`

- length < 10 → +0
- conventional prefix? no
- starts with capital? yes → +20
- noisy? yes (`update`) → -20

Per-commit score: 0

If many commits are titled like `Update/Changes`, the average message score will drop sharply.

1.8 Code health score (0–100) — optional “real code” analysis

This is enabled when:

- `RepositoryQualityAnalyzer.enable_code_health == True` AND
- analysis libraries are importable for that language (Radon / Esprima / Javalang)

The analyzer looks for source files in these languages:

- Python: `.py`
- TypeScript: `.ts`, `.tsx`
- JavaScript: `.js`, `.jsx`
- Java: `.java`

It collects file paths by walking directories (depth-limited) using the GitHub Contents API.

Configuration knobs:

- `code_health_max_files` (default 50 if passed as numeric; otherwise 30)
- `code_health_max_depth` (default 6 if passed as numeric; otherwise 4)

Hard safety limits:

- Ignores common large/vendor dirs (`node_modules`, `dist`, `build`, `.venv`, ...)
- Skips files larger than 120,000 characters

1.8.1 Python code health with Radon (preferred)

Function: `_analyze_python_code_health(contents)` when `radon` is available.

For each file:

- Maintainability Index (MI): `mi_visit(content, multi=True)`
- Cyclomatic complexity blocks: `cc_visit(content)`
- Compute `avg_cc` as mean block complexity for that file

Across files:

- `avg_mi = mean(mi_values)`
- `avg_cc = mean(avg_cc_values)`

Score:

- `base = clamp(avg_mi, 0, 100)`
- `cc_penalty = clamp(max(avg_cc - 6, 0) * 4.5, 0, 28)`
- `score = clamp(base - cc_penalty, 0, 100)`

So complexity only starts penalizing after average CC exceeds ~6.

1.8.2 Python code health fallback (no Radon)

If Radon is missing, the analyzer:

- Parses AST
- Estimates cyclomatic complexity by counting decision nodes (`if/for/while/try/match/...`) and boolean ops
- Penalizes large average file length

Score:

- start at 100
- subtract:
 - `cc_penalty = clamp(max(avg_cc - 6, 0) * 8, 0, 65)`
 - `size_penalty = clamp(max(avg_loc - 350, 0)/50 * 2, 0, 20)`

1.8.3 JS/TS code health (Esprima)

If `esprima` is available, it parses modules/scripts tolerantly and counts:

- If/for/while/catch/ternary/switch-case paths
- logical expressions `&&` and `||`

Then applies complexity + size penalties:

- threshold CC ≈ 6
- size threshold ≈ 450 LOC

1.8.4 Java code health (Javalang)

If `javalang` is available, it parses Java and counts decision nodes + boolean ops.

Then applies complexity + size penalties:

- threshold CC ≈ 8
- size threshold ≈ 550 LOC

1.8.5 Polyglot repos ("Mixed")

If multiple languages are analyzed successfully, the analyzer combines them via a capped file-count weighted average:

- `weight_i = clamp(files_analyzed_i, 1, 20)`
- `combined_score = sum(score_i*weight_i)/sum(weight_i)`

This prevents one language with many files from dominating too much, while still rewarding better coverage.

1.9 Per-repo total score (category-weighted)

Function: `_weighted_category_score(scores)`

- Collect (score, weight) pairs for the keys present in `scores`
- Compute:

```
total = clamp(sum(score_i * weight_i) / sum(weight_i), 0, 100)
```

Because it divides by the **sum of active weights**, missing categories do not drag the repo down artificially.

1.10 Portfolio overall score (across repos)

Function: `_calculate_weighted_score(scores, repos)`

Each repo gets an "importance" weight:

- `weight = 1 + stars*0.08 + recency_bonus`
- `recency_bonus` by last push/update:
 - ≤ 30 days: `+1.0`
 - ≤ 90 days: `+0.6`
 - ≤ 180 days: `+0.3`
 - else: `+0.0`
- cap: `weight  $\leq 5$`

Then:

```
overall = clamp(sum(repo_score_i * weight_i)/sum(weight_i), 0, 100)
```

1.11 Dynamic calibration and dynamic grade thresholds

After per-repo scoring, the analyzer adjusts strictness relative to the candidate's own portfolio:

1.11.1 Calibration (portfolio-relative blending)

Function: `_apply_dynamic_calibration(detailed_analysis)`

- Compute portfolio mean `avg` and std `std` over raw repo totals
- Compute a z-scaled score:

```
z_scaled = 50 + ((raw - avg)/std) * 15
```

- Blend:

```
calibrated = raw*0.75 + z_scaled*0.25 + 3
```

This nudges scores toward a portfolio-relative distribution (and adds a small +3 lift).

1.11.2 Dynamic thresholds

Function: `_derive_dynamic_thresholds(calibrated_scores)`

- Use calibrated portfolio average + std to compute A/B/C/D cutoffs
- Clamp to reasonable bounds
- Force monotonic order (A highest, then B, then C, ...)

This prevents "everyone gets F" on portfolios where all repos are small personal projects.

2) How RapidFuzz is used (rapidfuzz)

RapidFuzz is only used in **commit message repetition penalty**, to detect near-duplicate titles.

Function: `_compute_repetition_penalty(messages)`

Inputs:

- list of commit message first lines (lowercased, stripped)

Penalty is the sum of:

1. Exact duplicates

- `unique_ratio = unique_count / total_count`
- `exact_dup_penalty = (1 - unique_ratio) * 22`

2. Near duplicates (requires RapidFuzz)

- take at most 20 messages (to cap $O(n^2)$ cost)

- compute all pairwise `fuzz.ratio(a, b)`
- `high_similarity_ratio = (# pairs with ratio >= 88) / (# pairs)`
- `near_dup_penalty = high_similarity_ratio * 18`

Final repetition penalty:

- `penalty = clamp(exact_dup_penalty + near_dup_penalty, 0, 28)`

This penalty is subtracted from the average commit message score.

Worked micro-example (illustrative):

- messages: ["update", "update", "update readme", "update README"]
- exact duplicates are high → unique_ratio low → exact penalty increases
- RapidFuzz likely flags "update README" ~ "update readme" as near duplicates → near penalty adds more

3) How Textstat is used (textstat)

Textstat is used only in README scoring:

- metric: `textstat.flesch_reading_ease(readme_text)`
- converted into a readability bonus of 6–20 points

Operational notes:

- `textstat` is an **optional dependency**; if it fails import, readability bonus is skipped.
- Some older `textstat` builds depend on `pkg_resources` (historically provided by `setuptools`). Newer `setuptools` releases may no longer ship `pkg_resources`, which can break those older `textstat` versions.
- This repo pins `textstat==0.7.13` in `backend/requirements.txt`, which imports cleanly in modern Python environments.

4) How Radon helps Code Health (radon)

When available, Radon provides:

- **Maintainability Index (MI)**: a composite metric roughly tied to readability, complexity, and volume.
- **Cyclomatic complexity (CC)**: branching/paths complexity.

This analyzer uses MI as the base and applies a penalty only if average CC exceeds a threshold.

Worked example using your linked repository

Repository:

- <https://github.com/vihasreddy58/AI-Powered-Diagnostic-and-Treatment-Support-System-for-Medical-Professionals->

README inputs (computed from the public README)

From the current `README.md` content:

- `word_count` ≈ 156 ($\geq 120 \rightarrow +15$)
- features section detected ($\rightarrow +10$)
- contributing/development keywords detected ($\rightarrow +8$)
- install/setup detected: no ($\rightarrow +0$)
- usage/how-to detected: no ($\rightarrow +0$)
- code blocks: no ($\rightarrow +0$)
- badges: no ($\rightarrow +0$)

So structure score = $15 + 10 + 8 = 33$.

If `textstat` readability is available, it would add an additional 6–20 points.

Code Health inputs (computed from `app.py`)

Using Radon on `app.py` (public source):

- file length ≈ 592 lines
- MI ≈ 30.64
- average CC across blocks ≈ 3.0

Score calculation:

- `base` = 30.64
- `cc_penalty` = $\max(3.0 - 6.0, 0) * 4.5 = 0$
- `code_health_score` ≈ 30.64

Interpretation:

- Complexity is fine (CC), but MI is low $\rightarrow$ the file is likely hard to maintain (large, dense, or otherwise complex by MI's formula).

5) Contribution Analyzer (backend/services/contribution_analyzer.py)

The contribution analyzer is separate from the code quality analyzer.

5.1 What it collects

`analyze_contributions(repositories, username)` returns:

- `summary.total_commits`
- `summary.owned_repo_commits` (commits authored by the user in their own repos)
- `summary.fork_contributions` (commits authored by the user in forked repos)
- `summary.repos_with_commits`
- `summary.active_contributor = total_commits >= 50`

Plus:

- `commit_activity` (monthly + weekday distribution) from up to 10 owned repos
- `top_repositories` by user commits

- `contribution_streak` (consecutive months with activity)

5.2 Owned repositories analysis

- Select up to 15 owned repos by (stars, pushed_at)
- For each repo:
 - fetch up to 100 commits
 - keep only commits where `is_user == True`
 - compute ownership percentage: `user_commits / total_commits * 100`
 - classify commit message quality (simple rule: 10–72 chars and starts with capital or has ':' early)
 - determine if repo is active (pushed in last 90 days)

5.3 Commit activity over time

- For each of up to 10 repos:
 - fetch up to 50 commits
 - count user commits per month and weekday
- Return the latest 6 months of activity.

5.4 Streak calculation

- Uses `monthly_activity`
 - `current_streak_months` = number of most-recent months (in the available list) where activity > 0
 - `is_consistent = current_streak_months >= 3`
-

6) Tech Stack Analyzer (backend/services/tech_stack_analyzer.py)

6.1 Repository selection

- Filters out forks
- Scores repos by:
 - recency (updated within last year) + stars*10
- Analyzes up to 50 repos.

6.2 Detection methods

For each repo (depth-limited traversal up to 3 levels):

1. Primary language signal
 - If `repo.language` exists, add it with `confidence=90`.
2. File extension signals
 - Detect HTML/CSS/JS/TS/Python/Java/Jupyter by file endings.
3. Config-file signals
 - If files like `vite.config.js`, `tsconfig.json`, `tailwind.config.js`, etc exist, add corresponding tech.
4. Dependency manifest parsing

- If `package.json` exists:
 - parse dependencies + devDependencies
 - add Node.js, frameworks, and other tools based on package names
- If `requirements.txt` exists:
 - parse packages
 - map known packages → technologies (Flask/Django/FastAPI/TensorFlow/etc.)

Each detected technology stores:

- `confidence` (hard-coded per rule)
- `evidence` (human readable strings)
- `type` (language/framework/library/tool/concept/runtime/testing/database)

6.3 Usage frequency

At portfolio level:

- `usageFrequency = (count_of_repos_with_tech / repos_analyzed) * 100`

7) Skill Authenticity (backend/services/skill_matcher.py)

This is the dominant factor in the final score.

7.1 Matching rules

- Normalize skills (lowercase, strip punctuation, collapse whitespace)
- Canonicalize known variants (e.g., `reactjs` → `React`)
- Match resume skills to GitHub skills using:
 - exact match
 - canonical match
 - “simple fuzzy” match (substring containment or shared words)

Note: This fuzzy matching does **not** use RapidFuzz; it is a lightweight heuristic.

7.2 Authenticity score

Function: `_calculate_authenticity_score(matched_skills, missing_skills, extra_skills)`

- `base_score = (matched / (matched+missing)) * 100`
- extra bonus: `min(len(extra_skills)*2, 10)`
- final: `clamp(base_score + extra_bonus, 0, 100)`

8) Profile Signal (backend/services/scoring_engine.py)

Profile signal intentionally avoids social popularity metrics (followers).

Function: `_score_profile(github_data)`

Inputs:

- `github_data.profile` from GitHub API
- `github_data.contribution_stats` from `fetch_contribution_stats()`

Subscores:

1. Completeness score (0–100)

- 5 boolean fields: bio, name, location, company, blog
- $\text{completeness} = (\# \text{ present} / 5) * 100$

2. Depth score (0–100)

- $\text{min}(100, \text{public_repos}/25*100)$

3. Active volume score (0–100)

- $\text{min}(100, \text{active_repos}/8*100)$

4. Activity consistency score (0–100)

- $\text{min}(100, \text{active_repos}/\text{base_repos}*100)$
 - `base_repos` = `total_repositories` or `public_repos`

5. Account maturity score (0–100)

- based on account age in days
- saturates at ~4 years (1460 days)

Final profile signal:

- $0.20*\text{completeness} + 0.25*\text{depth} + 0.30*\text{active_volume} + 0.15*\text{consistency} + 0.10*\text{maturity}$

Worked example (illustrative)

Assume:

- Completeness: 4 of 5 fields present → `completeness` = 80
- `public_repos` = 12 → `depth` = $\text{min}(100, 12/25*100) = 48$
- `active_repos` = 5 → `active_volume` = $\text{min}(100, 5/8*100) = 62.5$
- `total_repositories` = 20 → `consistency` = $\text{min}(100, 5/20*100) = 25$
- Account age: ~2 years (~730 days) → `maturity` = $\text{min}(100, 730/1460*100) = 50$

Then:

```
profile_signal = 0.20*80 + 0.25*48 + 0.30*62.5 + 0.15*25 + 0.10*50
= 16 + 12 + 18.75 + 3.75 + 5 = 55.5
```

9) Tech stack score (backend/services/scoring_engine.py)

Function: `_score_tech_stack(tech_stack_data)`

- `breadth = min(len(detected_skills), 25)/25*100`
- `avg_confidence = average of all detected skill confidences`
- `stack_score = 0.6*breadth + 0.4*avg_confidence`

Worked example (illustrative)

Assume the analyzer detected 12 distinct technologies in `detected_skills`, with average confidence 78:

- `breadth = 12/25*100 = 48`
- `avg_confidence = 78`

Then:

```
tech_stack = 0.6*48 + 0.4*78 = 28.8 + 31.2 = 60.0
```

10) Commit activity score (backend/services/scoring_engine.py)

Function: `_score_commit_activity(contributions)`

Inputs:

- `contributions.summary.total_commits`
- `contributions.commit_activity.total_recent_commits` (latest 6 months window in analyzer)

Score:

- `recent_score = min(100, recent_commits * 4)`
 - 25 recent commits → 100
- `volume_score = min(100, total_commits/200*100)`
- `commit_activity_score = 0.6*recent_score + 0.4*volume_score`

Worked example (illustrative)

Assume:

- `total_commits = 140`
- `recent_commits (last 6 months) = 18`

Then:

- `recent_score = min(100, 18*4) = 72`
- `volume_score = min(100, 140/200*100) = 70`

```
commit_activity = 0.6*72 + 0.4*70 = 43.2 + 28 = 71.2
```

11) Overall score and rating (backend/services/scoring_engine.py)

Weights (sum = 1.0):

- `skill_authenticity: 0.60`
- `code_quality: 0.10`

- commit_activity: 0.10
- tech_stack: 0.10
- profile_signal: 0.10

Notes:

- skill_authenticity is clamped so it is **never lower** than match_percentage.

Final overall:

$overall = \sum(component_score * weight)$

Worked example (illustrative)

Assume the scoring engine receives:

- skill_authenticity = 76
- code_quality = 62
- commit_activity = 71.2
- tech_stack = 60
- profile_signal = 55.5

Then:

$overall = 0.60*76 + 0.10*62 + 0.10*71.2 + 0.10*60 + 0.10*55.5$

$= 45.6 + 6.2 + 7.12 + 6 + 5.55 = 70.47$

Rating:

- 70.47 maps to **Very Good** (because it is ≥ 70 and < 85).

Rating buckets:

-  = 85: Excellent
-  = 70: Very Good
-  = 55: Good
-  = 40: Average
- else: Below Average

12) How these show up in the report (backend/services/report_generator.py)

Key report sections:

- **codeQualityReport**
 - overallScore, grade, metrics, categoryScores, strengths/weaknesses/suggestions
- **contributionReport**
 - summary, topRepositories, commitActivity, streak

- `technicalAnalysis`
 - `detectedSkills` and categories
- `skillMatchingReport`
 - `matchedSkills/missingSkills/extraSkills` + `authenticityScore`
- `scoringAnalysis`
 - `overallScore` + breakdown + rating

The frontend dashboard renders these fields directly, so changing formulas impacts the UI output.